In [1]:
```
import pandas as pd
import numpy as np

df=pd.read_csv('Social_Network_Ads.csv')
df.head()
```

Out[1]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19.0	19000.0	0
1	15810944	Male	35.0	20000.0	0
2	15668575	Female	26.0	43000.0	0
3	15603246	Female	27.0	57000.0	0
4	15804002	Male	19.0	76000.0	0

In [2]:
```
df.shape
```

Out[2]:
```
(400, 5)
```

In [3]:
```
df.describe()
```

Out[3]:

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

In [4]:
```
df.isnull().sum()
```

Out[4]:
```
User ID            0
Gender             0
Age                0
EstimatedSalary    0
Purchased          0
dtype: int64
```

In [5]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
```

In [6]:
```
df['Gender'] = df['Gender'].map({'Male': 1, 'Female': 0})
```

In [7]:
```
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']
```

In [8]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random
```

In [9]:
```
df['Gender']=df['Gender'].map({'Male':1,'Female':0})
```

In [10]:
```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [11]:
```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Out[11]:
```
LogisticRegression()
```

In [12]:
```
y_pred = model.predict(X_test)
```

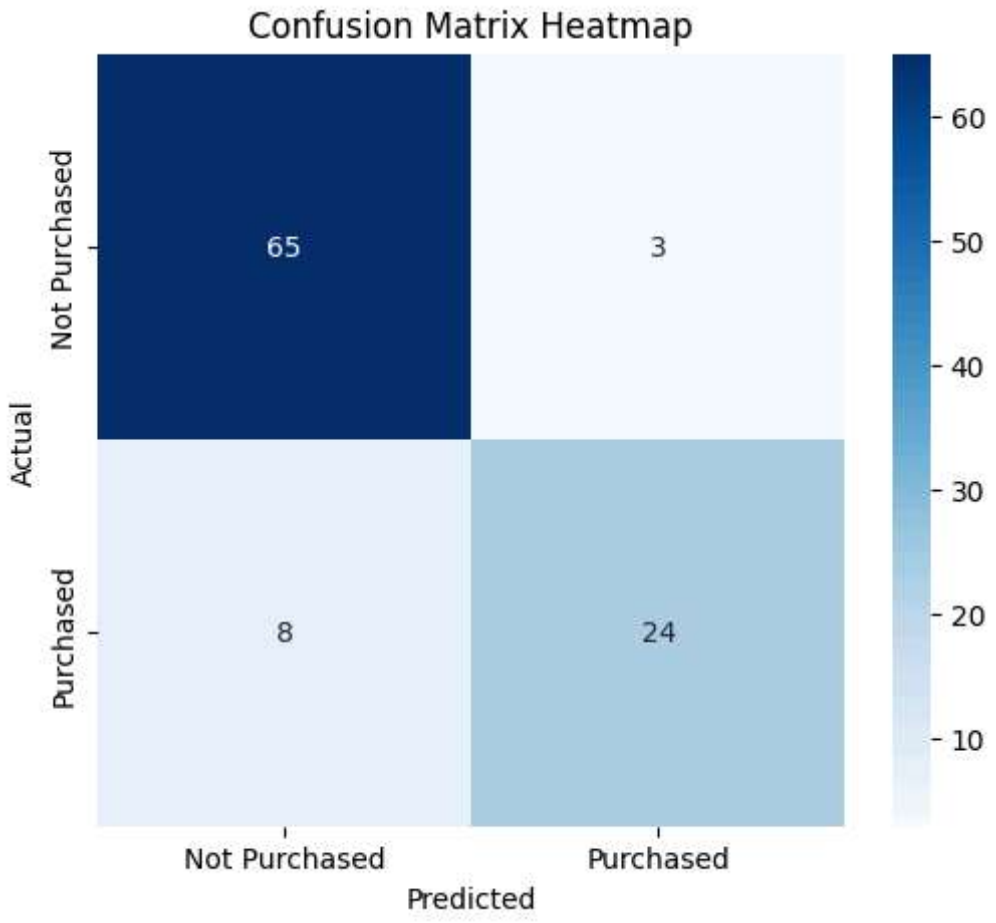
In [13]:
```
cm = confusion_matrix(y_test, y_pred)
```

In [14]:
```
TN, FP, FN, TP = cm.ravel()

# Metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
error_rate = 1 - accuracy
precision = TP / (TP + FP) if (TP + FP) != 0 else 0
recall = TP / (TP + FN) if (TP + FN) != 0 else 0

print("Confusion Matrix:")
print(cm, "\n")

print(f"True Positives (TP): {TP}")
print(f"False Positives (FP): {FP}")
print(f"True Negatives (TN): {TN}")
print(f"False Negatives (FN): {FN}\n")

print(f"Accuracy: {accuracy:.4f}")
print(f"Error Rate: {error_rate:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
```

```
Confusion Matrix:
[[65  3]
 [ 8 24]]

True Positives (TP): 24
False Positives (FP): 3
True Negatives (TN): 65
False Negatives (FN): 8

Accuracy: 0.8900
Error Rate: 0.1100
Precision: 0.8889
Recall: 0.7500
```

In [15]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [16]:
```
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Purchased', 'Purchased'],
            yticklabels=['Not Purchased', 'Purchased'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix Heatmap')
plt.show()
```



In [ ]: